

USING HERMES

Table des matières

1. Interface CLI

- Lancement de la CLI
- Mise en page & barre de statut
- Raccourcis clavier
- Commandes slash & commandes rapides
- Personnalités & saisie multiligne
- Interruption & sessions en arrière-plan

2. Interface TUI

- Lancement & prérequis
- Avantages du TUI
- Raccourcis & commandes slash
- Configuration & sessions

3. Configuration

- Structure des répertoires
- Gestion de la configuration
- Backends de terminal
- Compression de contexte & modèles auxiliaires
- Paramètres d'affichage & autres options

4. Sessions

- Fonctionnement des sessions
- Reprendre une session CLI
- Nommage & gestion des sessions
- Recherche dans les sessions
- Stockage & nettoyage

5. Profils

- Qu'est-ce qu'un profil ?
- Créer & utiliser un profil
- Gateways multi-profils

6. Git Worktrees

- Pourquoi utiliser les worktrees ?
- Démarrage rapide
- Agents en parallèle
- Mode automatique hermes -w

7. Docker

- Exécuter Hermes dans Docker
- Mode gateway & dashboard
- Volumes & multi-profils

- Connexion aux serveurs d'inférence locaux

8. Sécurité

- Modèle de sécurité
- Approbation des commandes dangereuses
- Autorisation des utilisateurs (gateway)
- Isolation par conteneur
- Protection SSRF & analyse Tirth

9. Checkpoints & Rollback

- Ce qui déclenche un checkpoint
- Lister & prévisualiser
- Restaurer avec /rollback
- Configuration & meilleures pratiques

1. Interface CLI

La CLI Hermes est une interface terminal complète (TUI classique) — pas une interface web. Elle propose l'édition multiligne, l'autocomplétion des commandes slash, l'historique de conversation, l'interruption-redirection et l'affichage de la progression des outils en temps réel. Conçue pour ceux qui vivent dans le terminal.

1.1 Lancement de la CLI

```
# Session interactive (défaut)
hermes

# Mode requête unique (non interactif)
hermes chat -q "Bonjour"

# Avec un modèle spécifique
hermes chat --model "anthropic/claude-sonnet-4"

# Avec un fournisseur spécifique
hermes chat --provider nous # Nous Portal
hermes chat --provider openrouter # Force OpenRouter

# Reprendre des sessions précédentes
hermes --continue # Reprendre la session CLI la plus récente
hermes --resume <session_id> # Reprendre une session par ID

# Mode worktree git isolé
hermes -w # Mode interactif dans un worktree
hermes -w -q "Fix issue #123" # Requête unique dans un worktree
```

1.2 Barre de statut

Une barre de statut persistante s'affiche au-dessus de la zone de saisie et se met à jour en temps réel :

■ claude-sonnet-4-20250514 ■ 12.4K/200K ■ [■■■■■■■■■■] 6% ■ \$0.06 ■ 15m

Élément	Description
Nom du modèle	Modèle actuel (tronqué si > 26 caractères)
Compteur de tokens	Tokens de contexte utilisés / fenêtre max
Barre de contexte	Indicateur visuel avec seuils de couleur
Coût	Coût estimé de la session
Durée	Temps de session écoulé

Codage couleur du contexte :

Couleur	Seuil	Signification
Vert	< 50 %	Beaucoup d'espace disponible

Jaune	50–80 %	Commence à se remplir
Orange	80–95 %	Approche la limite
Rouge	≥ 95 %	Proche du dépassement — envisager /compress

1.3 Raccourcis clavier

Touche	Action
Enter	Envoyer le message
Alt+Enter ou Ctrl+J	Nouvelle ligne (saisie multiligne)
Alt+V	Coller une image depuis le presse-papiers
Ctrl+V	Coller du texte (avec détection d'image)
Ctrl+G	Ouvrir le buffer de saisie dans \$EDITOR
Ctrl+C	Interrompre l'agent (double appui < 2s pour quitter)
Ctrl+D	Quitter
Ctrl+Z	Suspendre Hermes en arrière-plan (Unix uniquement)
Tab	Accepter la suggestion automatique ou compléter

1.4 Commandes slash

Tapez / pour afficher le menu d'autocomplétion. Hermes dispose d'un grand ensemble de commandes.

Commande	Description
/help	Afficher l'aide des commandes
/model	Afficher ou changer le modèle actuel
/tools	Lister les outils disponibles
/skills browse	Parcourir le hub de skills
/background	Lancer une requête en session en arrière-plan
/skin	Afficher ou changer le thème CLI actif
/voice on	Activer le mode vocal CLI
/reasoning high	Augmenter l'effort de raisonnement
/title Mon Session	Nommer la session courante
/compress	Compresser manuellement le contexte
/yolo	Basculer le mode YOLO (approbation auto de toutes les commandes)

1.5 Commandes rapides personnalisées

Définissez des commandes personnalisées qui exécutent des commandes shell instantanément sans invoquer le LLM.

```
# ~/.hermes/config.yaml
quick_commands:
  status:
    type: exec
    command: systemctl status hermes-agent
  gpu:
    type: exec
    command: nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv,noheader
  restart:
    type: alias
    target: /gateway restart
```

1.6 Personnalités

Définissez une personnalité prédéfinie pour changer le ton de l'agent :

```
/personality pirate
/personality concise
/personality teacher
```

Personnalités intégrées : *helpful, concise, technical, creative, teacher, kawaii, catgirl, pirate, shakespeare, surfer, noir, uwu, philosopher, hype*. Vous pouvez aussi définir des personnalités personnalisées dans `~/.hermes/config.yaml`.

1.7 Interrompre l'agent

Vous pouvez interrompre l'agent à tout moment. Le comportement est contrôlé par `display.busy_input_mode` :

Mode	Comportement
interrupt (défaut)	Votre message interrompt l'opération en cours immédiatement
queue	Votre message est mis en file d'attente et envoyé après la fin de l'agent
steer	Votre message est injecté dans l'exécution courante après le prochain appel d'outil

1.8 Sessions en arrière-plan

Lancez une requête dans une session en arrière-plan séparée pendant que vous continuez à utiliser la CLI :

```
/background Analyser les logs dans /var/log et résumer les erreurs du jour
```

Chaque commande **/background** démarre une session agent complètement séparée dans un thread démon. La session de premier plan reste entièrement interactive. Le résultat apparaît sous forme de panneau dans votre terminal une fois la tâche terminée.

2. Interface TUI

Le TUI est l'interface moderne de Hermes — une interface terminal basée sur Node.js, utilisant le même runtime Python que la CLI classique. Même agent, mêmes sessions, mêmes commandes slash ; une surface plus propre et plus réactive. C'est la façon recommandée d'utiliser Hermes de manière interactive.

2.1 Lancement

```
# Lancer le TUI
hermes --tui

# Reprendre la dernière session TUI
hermes --tui -c

# Reprendre une session spécifique
hermes --tui -r 20260409_000000_aa11bb
hermes --tui --resume "ma session"

# Activation via variable d'environnement
export HERMES_TUI=1
hermes # utilise maintenant le TUI
```

2.2 Prérequis

Node.js ≥ 20 est requis — le TUI s'exécute en tant que sous-processus lancé depuis la CLI Python. La commande **hermes doctor** vérifie cette condition. Lors du premier lancement, Hermes installe les dépendances Node du TUI dans *ui-tui/node_modules* (une seule fois, quelques secondes). Les lancements suivants sont rapides.

2.3 Avantages du TUI par rapport à la CLI classique

- **Premier frame instantané** — la bannière s'affiche avant la fin du chargement.
- **Saisie non bloquante** — tapez et mettez en file vos messages avant que la session soit prête.
- **Overlays enrichis** — sélecteur de modèle, de session, invites d'approbation sous forme de panneaux modaux.
- **Panneau de session en direct** — outils et skills se remplissent progressivement.
- **Sélection à la souris** — surlignez avec un arrière-plan uniforme.
- **Rendu en écran alternatif** — mises à jour différentielles sans scintillement.

2.4 Statut en temps réel

Statut	Signification
starting agent...	ID de session actif ; outils et skills en cours d'initialisation
ready	L'agent est inactif, accepte les entrées
thinking... / running...	L'agent raisonne ou exécute un outil
interrupted	Le tour actuel a été annulé ; appuyez sur Entrée pour renvoyer

forging session... /
resuming...

Connexion initiale ou reprise via --resume

2.5 Configuration spécifique au TUI

```
display:  
skin: default # skin CLI intégré ou personnalisé  
personality: helpful  
details_mode: collapsed # hidden | collapsed | expanded  
sections:  
  thinking: expanded # toujours ouvert  
  tools: expanded # toujours ouvert  
  activity: collapsed # opt-in pour le panneau d'activité  
mouse_tracking: true # désactiver si conflit terminal
```

■ Les sessions sont partagées entre le TUI et la CLI classique — les deux écrivent dans le même `~/hermes/state.db`. Vous pouvez démarrer une session dans l'un et la reprendre dans l'autre.

3. Configuration

Tous les paramètres sont stockés dans le répertoire `~/.hermes/` pour un accès facile.

3.1 Structure des répertoires

```
~/.hermes/  
■■■ config.yaml # Paramètres (modèle, terminal, TTS, compression, etc.)  
■■■ .env # Clés API et secrets  
■■■ auth.json # Identifiants OAuth  
■■■ SOUL.md # Identité primaire de l'agent (slot #1 du prompt système)  
■■■ memories/ # Mémoire persistante (MEMORY.md, USER.md)  
■■■ skills/ # Skills créées par l'agent  
■■■ cron/ # Tâches planifiées  
■■■ sessions/ # Sessions de gateway  
■■■ logs/ # Logs (erreurs, gateway – secrets auto-expurgés)
```

3.2 Gestion de la configuration

```
hermes config # Voir la configuration actuelle  
hermes config edit # Ouvrir config.yaml dans votre éditeur  
hermes config set KEY VAL # Définir une valeur spécifique  
hermes config check # Vérifier les options manquantes  
hermes config migrate # Ajouter les options manquantes de manière interactive  
  
# Exemples :  
hermes config set model anthropic/claude-opus-4  
hermes config set terminal.backend docker  
hermes config set OPENROUTER_API_KEY sk-or-... # Sauvegarde dans .env
```

3.3 Priorité de configuration

Les paramètres sont résolus dans cet ordre (priorité décroissante) :

1. Arguments CLI (ex. `hermes chat --model ...`)
2. `~/.hermes/config.yaml` — fichier de configuration principal
3. `~/.hermes/.env` — pour les variables d'environnement ; **requis** pour les secrets
4. Valeurs par défaut intégrées

3.4 Backends de terminal

Hermes supporte sept backends de terminal. Chacun détermine où les commandes shell s'exécutent réellement.

Backend	Exécution	Isolation	Idéal pour
local	Votre machine	Aucune	Développement, usage personnel
docker	Conteneur Docker persistant	Complète	Sandbox sécurisé, CI/CD

ssh	Serveur distant via SSH	Frontière réseau	Dev à distance
modal	Sandbox cloud Modal	Complète (VM cloud)	Calcul cloud éphémère
daytona	Workspace Daytona	Complète (conteneur)	Environnements cloud gérés
vercel_sandbox	microVM Vercel Sandbox	Complète (microVM)	Exécution cloud persistante
singularity	Conteneur Singularity/Apptainer	Espaces de noms	Clusters HPC

Configuration du backend Docker

```
terminal:
backend: docker
docker_image: "nikolaik/python-nodejs:python3.11-nodejs20"
docker_mount_cwd_to_workspace: false # Monter le répertoire de lancement dans /workspace
docker_run_as_host_user: false # Exécuter comme l'utilisateur hôte
docker_forward_env: # Variables d'env à transmettre
- "GITHUB_TOKEN"
docker_volumes: # Montages de répertoires hôte
- "/home/user/projects:/workspace/projects"
container_cpu: 1 # Cœurs CPU
container_memory: 5120 # Mo
container_persistent: true # Persistance du filesystem
```

3.5 Compression de contexte

Hermes compresse automatiquement les longues conversations pour rester dans la fenêtre de contexte du modèle.

```
compression:
enabled: true
threshold: 0.50 # Compresser à 50% de la limite de contexte

auxiliary:
compression:
model: "google/gemini-3-flash-preview" # Modèle pour la synthèse
provider: "auto"
```

3.6 Modèles auxiliaires

Hermes utilise des modèles auxiliaires pour les tâches secondaires (analyse d'images, synthèse de pages web, génération de titres de session, compression de contexte). Par défaut, toutes les tâches auxiliaires utilisent votre modèle de chat principal.

```
auxiliary:
vision:
provider: "openrouter"
model: "openai/gpt-4o"
web_extract:
provider: "auto"
model: "google/gemini-2.5-flash"
```

3.7 Paramètres d'affichage

```
display:
tool_progress: all # off | new | all | verbose
skin: default # Thème CLI intégré ou personnalisé
streaming: false # Diffuser les tokens en temps réel
show_cost: false # Afficher le coût estimé dans la barre de statut
bell_on_complete: false # Sonnerie terminale à la fin de l'agent
resume_display: full # full (afficher les messages précédents) | minimal
language: fr # Langue de l'UI (en | zh | ja | de | es | fr | tr | uk)
```

4. Sessions

Hermes Agent sauvegarde automatiquement chaque conversation en tant que session. Les sessions permettent la reprise de conversation, la recherche inter-sessions et la gestion complète de l'historique des conversations.

4.1 Fonctionnement des sessions

Chaque conversation — CLI, Telegram, Discord, Slack, WhatsApp, Signal, Matrix et toute autre plateforme — est stockée sous forme de session avec l'historique complet des messages. Les sessions sont suivies dans deux systèmes complémentaires :

- Base de données SQLite (`~/hermes/state.db`) — métadonnées structurées avec recherche FTS5
- Transcriptions JSONL (`~/hermes/sessions/`) — transcriptions brutes incluant les appels d'outils

4.2 Reprendre une session CLI

```
# Reprendre la session CLI la plus récente
hermes --continue
hermes -c

# Reprendre une session nommée
hermes -c "mon projet"

# Reprendre par ID ou titre
hermes --resume 20250305_091523_a1b2c3d4
hermes --resume "refactoring auth"
hermes -r 20250305_091523_a1b2c3d4
```

■ Lors de la reprise, Hermes affiche un récapitulatif compact de la conversation précédente dans un panneau stylisé avant l'invite de saisie. Pour désactiver : `display.resume_display: minimal` dans `config.yaml`.

4.3 Nommage des sessions

Hermes génère automatiquement un titre court (3 à 7 mots) pour chaque session après le premier échange. Vous pouvez également définir un titre manuellement :

```
/title mon projet de recherche

# Depuis la ligne de commande
hermes sessions rename 20250305_091523_a1b2c3d4 "refactoring module auth"
```

Lorsqu'une session est compressée, Hermes crée une session de continuation avec un titre numéroté : `"mon projet"` → `"mon projet #2"` → `"mon projet #3"`.

4.4 Commandes de gestion des sessions

```
hermes sessions list # Lister les sessions récentes
hermes sessions list --source telegram # Filtrer par plateforme
hermes sessions export backup.jsonl # Exporter toutes les sessions
hermes sessions delete <id> # Supprimer une session
hermes sessions rename <id> <titre> # Renommer une session
hermes sessions prune # Supprimer les sessions > 90 jours
hermes sessions stats # Statistiques des sessions
```

4.5 Recherche dans les sessions

L'agent dispose d'un outil intégré **session_search** qui effectue une recherche en texte intégral dans toutes les conversations passées via le moteur FTS5 de SQLite. La syntaxe de requête FTS5 est supportée : mots-clés simples, phrases exactes entre guillemets, opérateurs booléens (OR, NOT), préfixes avec *.

4.6 Stockage & nettoyage

Quoi	Chemin	Description
Base SQLite	~/.hermes/state.db	Toutes les métadonnées + messages avec FTS5
Transcriptions gateway	~/.hermes/sessions/	Transcriptions JSONL par session
Index gateway	~/.hermes/sessions/sessions.json	Mappage clés → IDs de sessions

Nettoyage automatique opt-in :

```
sessions:
  auto_prune: true # opt-in – false par défaut
  retention_days: 90 # conserver les sessions terminées N jours
  vacuum_after_prune: true # récupérer l'espace disque après nettoyage
  min_interval_hours: 24 # ne pas relancer plus souvent que ça
```

5. Profils — Exécuter plusieurs agents

Exécutez plusieurs agents Hermes indépendants sur la même machine — chacun avec sa propre configuration, clés API, mémoire, sessions, skills et état de gateway.

5.1 Qu'est-ce qu'un profil ?

Un profil est un répertoire home Hermes séparé. Chaque profil possède son propre *config.yaml*, *.env*, *SOUL.md*, mémoires, sessions, skills, tâches cron et base de données d'état. Les profils permettent d'exécuter des agents séparés pour différents usages (assistant de code, bot personnel, agent de recherche) sans mélanger les états.

5.2 Démarrage rapide

```
hermes profile create coder # crée le profil + alias de commande "coder"
coder setup # configurer les clés API et le modèle
coder chat # démarrer une conversation
```

5.3 Créer un profil

Option	Commande	Effet
Profil vide	hermes profile create mybot	Nouveau profil avec skills intégrées
Cloner la config	hermes profile create work --clone	Copie config.yaml, .env et SOUL.md
Tout cloner	hermes profile create backup --clone-all	Snapshot complet (config + sessions + mémoire)
Cloner depuis un profil	hermes profile create work --clone --clone-from coder	Clone depuis un profil spécifique

5.4 Utiliser les profils

```
# Via l'alias de commande créé automatiquement
coder chat
coder setup
coder gateway start

# Via le flag -p
hermes -p coder chat
hermes --profile=coder doctor

# Définir un profil par défaut
hermes profile use coder
hermes chat # cible maintenant coder
hermes profile use default # revenir au profil par défaut
```

5.5 Gestion des profils

```
hermes profile list # Afficher tous les profils
hermes profile show coder # Détails d'un profil
hermes profile rename coder dev-bot # Renommer
hermes profile export coder # Exporter vers coder.tar.gz
hermes profile import coder.tar.gz # Importer depuis une archive
hermes profile delete coder # Supprimer (avec confirmation)
```

■■ Vous ne pouvez pas supprimer le profil par défaut (~/.hermes). Pour tout supprimer, utilisez **hermes uninstall**.

6. Git Worktrees

Hermes Agent est souvent utilisé sur de grands dépôts à long terme. Lorsque vous souhaitez exécuter plusieurs agents en parallèle sur le même projet, ou garder des refactorisations expérimentales isolées de votre branche principale, les **worktrees Git** sont la façon la plus sûre de donner à chaque agent son propre checkout sans dupliquer l'intégralité du dépôt.

6.1 Pourquoi utiliser les worktrees avec Hermes ?

Hermes traite le **répertoire de travail courant** comme la racine du projet. Si vous exécutez plusieurs agents dans le **même checkout**, leurs modifications peuvent interférer : un agent peut supprimer ou réécrire des fichiers que l'autre utilise.

Avec les worktrees, chaque agent obtient :

- Sa **propre branche et répertoire de travail**
- Son **propre historique de Checkpoint Manager** pour /rollback

6.2 Démarrage rapide

```
# Depuis la racine du dépôt principal
cd /chemin/vers/votre/depot

# Créer une nouvelle branche et un worktree dans ../repo-feature
git worktree add ../repo-feature feature/hermes-experiment

# Se déplacer dans le nouveau worktree et lancer Hermes
cd ../repo-feature
hermes
```

6.3 Exécuter plusieurs agents en parallèle

```
cd /chemin/vers/votre/depot

git worktree add ../repo-experiment-a feature/hermes-a
git worktree add ../repo-experiment-b feature/hermes-b

# Terminal 1
cd ../repo-experiment-a && hermes

# Terminal 2
cd ../repo-experiment-b && hermes
```

6.4 Mode automatique hermes -w

Hermes dispose d'un flag **-w** intégré qui **crée automatiquement un worktree git jetable** avec sa propre branche. Pas besoin de configurer les worktrees manuellement :

```
cd /chemin/vers/votre/depot
hermes -w

# Combiné avec une requête unique
hermes -w -q "Corriger l'issue #123"

# Pour des agents en parallèle : ouvrir plusieurs terminaux
# et lancer hermes -w dans chacun
```

6.5 Nettoyage des worktrees

```
# Supprimer le worktree et sa référence
git worktree remove ../repo-feature

# Note : les branches ne sont pas supprimées automatiquement
# Utiliser git branch pour gérer les branches manuellement
```

7. Docker

Il existe deux façons distinctes dont Docker s'intègre avec Hermes Agent : **(1)** exécuter Hermes lui-même dans Docker (focus principal de cette section) et **(2)** utiliser Docker comme backend de terminal (voir section Configuration).

7.1 Démarrage rapide

```
mkdir -p ~/.hermes
docker run -it --rm \
-v ~/.hermes:/opt/data \
nousresearch/hermes-agent setup
```

7.2 Mode gateway (arrière-plan persistant)

```
docker run -d \
--name hermes \
--restart unless-stopped \
-v ~/.hermes:/opt/data \
-p 8642:8642 \
nousresearch/hermes-agent gateway run
```

Le port 8642 expose le serveur API compatible OpenAI du gateway. Il est optionnel si vous n'utilisez que des plateformes de messagerie, mais requis pour le dashboard ou les outils externes.

7.3 Dashboard web

```
docker run -d \
--name hermes \
--restart unless-stopped \
-v ~/.hermes:/opt/data \
-p 8642:8642 \
-p 9119:9119 \
-e HERMES_DASHBOARD=1 \
nousresearch/hermes-agent gateway run
```

7.4 Docker Compose

```

services:
hermes:
image: nousresearch/hermes-agent:latest
container_name: hermes
restart: unless-stopped
command: gateway run
ports:
- "8642:8642" # API gateway
- "9119:9119" # dashboard
volumes:
- ~/.hermes:/opt/data
environment:
- HERMES_DASHBOARD=1
deploy:
resources:
limits:
memory: 4G
cpus: "2.0"

```

7.5 Ressources recommandées

Ressource	Minimum	Recommandé
Mémoire	1 Go	2–4 Go
CPU	1 cœur	2 cœurs
Disque (volume de données)	500 Mo	2+ Go (croît avec sessions/skills)

7.6 Connexion aux serveurs d'inférence locaux

Pour connecter Hermes à vLLM, Ollama ou d'autres serveurs locaux :

```

# macOS / Windows : utiliser host.docker.internal
model:
provider: custom
model: my-model
base_url: http://host.docker.internal:8000/v1
api_key: "none"

# Linux (réseau hôte)
docker run -d --network host ...
# puis dans config.yaml :
base_url: http://127.0.0.1:8000/v1

```

7.7 Dépannage courant

Problème	Solution
Le conteneur quitte immédiatement	Vérifier les logs : <code>docker logs hermes</code> . Fichier <code>.env</code> manquant ou invalide.
Erreurs de permission	Le conteneur utilise UID 10000. Configurer <code>HERMES_UID/HERMES_GID</code> ou <code>chmod -R 755 ~/.hermes</code>

Outils browser non fonctionnels	Playwright a besoin de mémoire partagée : ajouter --shm-size=1g
Gateway ne se reconnecte pas	La flag --restart unless-stopped gère la plupart des pannes. Sinon : docker restart hermes

8. Sécurité

Hermes Agent est conçu avec un modèle de sécurité en profondeur (defense-in-depth). Cette section couvre chaque couche de sécurité.

8.1 Vue d'ensemble du modèle de sécurité

- **Couche 1** — Autorisation des utilisateurs — listes d'autorisation, jumelage DM
- **Couche 2** — Approbation des commandes dangereuses — intervention humaine pour les opérations destructrices
- **Couche 3** — Isolation par conteneur — sandboxing Docker/Singularity/Modal avec paramètres renforcés
- **Couche 4** — Filtrage des identifiants MCP — isolation des variables d'environnement pour les sous-processus MCP
- **Couche 5** — Analyse des fichiers de contexte — détection d'injection de prompt dans les fichiers projet
- **Couche 6** — Isolation inter-sessions — les sessions ne peuvent pas accéder aux données des autres
- **Couche 7** — Assainissement des entrées — validation des paramètres de répertoire de travail

8.2 Approbation des commandes dangereuses

Avant d'exécuter une commande, Hermes la vérifie par rapport à une liste de patterns dangereux. Trois modes sont disponibles :

Mode	Comportement
manual (défaut)	Toujours demander l'approbation de l'utilisateur pour les commandes dangereuses
smart	Utiliser un LLM auxiliaire pour évaluer le risque. Les commandes à faible risque sont auto-approuvées.
off	Désactiver toutes les vérifications d'approbation (équivalent à --yolo)

```
approvals:
mode: manual # manual | smart | off
timeout: 60 # secondes d'attente pour la réponse
```

Mode YOLO

Le mode YOLO contourne **toutes** les invites d'approbation de commandes dangereuses pour la session courante. Il peut être activé via le flag `--yolo`, la commande `/yolo`, ou la variable d'environnement `HERMES_YOLO_MODE=1`.

■■ Le mode YOLO désactive TOUTES les vérifications de sécurité pour la session. Utilisez-le uniquement dans des environnements de confiance. La liste noire (blocklist) reste toujours active.

8.3 Autorisation des utilisateurs (gateway)

Lors de l'exécution du gateway de messagerie, Hermes contrôle qui peut interagir avec le bot via un système d'autorisation en couches :

```
# Dans ~/.hermes/.env
TELEGRAM_ALLOWED_USERS=123456789,987654321
DISCORD_ALLOWED_USERS=111222333444555666
GATEWAY_ALLOWED_USERS=123456789 # Multi-plateforme
GATEWAY_ALLOW_ALL_USERS=true # Accès ouvert (avec précaution !)
```

Système de jumelage DM

Pour une autorisation plus flexible, Hermes inclut un système de jumelage par code : les utilisateurs inconnus reçoivent un code de jumelage à 8 caractères, que le propriétaire du bot approuve via la CLI.

```
hermes pairing list # Lister les utilisateurs en attente et approuvés
hermes pairing approve telegram ABC12DEF # Approuver un code
hermes pairing revoke telegram 123456789 # Révoquer l'accès
```

8.4 Isolation par conteneur Docker

Chaque conteneur s'exécute avec un renforcement de sécurité strict :

Flag	Effet
--cap-drop ALL	Supprimer toutes les capacités Linux
--cap-add DAC_OVERRIDE, CHOWN, FOWNER	Uniquement les capacités minimales nécessaires
--security-opt no-new-privileges	Bloquer l'escalade de privilèges
--pids-limit 256	Limiter le nombre de processus
--tmpfs /tmp (512Mo)	/tmp de taille limitée

8.5 Protection SSRF

Tous les outils capables d'accéder à des URL (recherche web, extraction web, vision, browser) valident les URL avant de les récupérer pour prévenir les attaques SSRF. Les adresses bloquées incluent les réseaux privés (RFC 1918), loopback, link-local, CGNAT et les endpoints de métadonnées cloud.

8.6 Meilleures pratiques pour le déploiement en production

- Définir des listes d'autorisation explicites — ne jamais utiliser `GATEWAY_ALLOW_ALL_USERS=true` en production
- Utiliser le backend conteneur — définir `terminal.backend: docker` dans `config.yaml`
- Restreindre les limites de ressources — CPU, mémoire et disque appropriés
- Stocker les secrets de manière sécurisée — clés API dans `~/.hermes/.env` avec permissions correctes (chmod 600)
- Activer le jumelage DM — utiliser des codes de jumelage plutôt que des IDs en dur
- Auditer régulièrement la liste d'autorisation de commandes — `command_allowlist` dans `config.yaml`
- Définir `MESSAGING_CWD` — ne pas laisser l'agent opérer depuis des répertoires sensibles
- Exécuter en non-root — ne jamais lancer le gateway en tant que root
- Surveiller les logs — vérifier `~/.hermes/logs/` pour les tentatives d'accès non autorisées

- Maintenir à jour — exécuter hermes update régulièrement pour les correctifs de sécurité

9. Checkpoints & Rollback

Hermes Agent capture automatiquement votre projet avant les **opérations destructrices** et vous permet de le restaurer avec une seule commande. Les checkpoints sont **activés par défaut** — aucun coût quand aucun outil de modification de fichiers ne s'exécute. Ce filet de sécurité est alimenté par un **Checkpoint Manager** interne qui maintient un dépôt git fantôme séparé sous `~/.hermes/checkpoints/` — votre vrai `.git` n'est jamais touché.

9.1 Ce qui déclenche un checkpoint

Les checkpoints sont pris automatiquement avant :

- **Outils de fichiers** — `write_file` et `patch`
- **Commandes terminal destructrices** — `rm`, `mv`, `sed -i`, `truncate`, `shred`, redirections (`>`), `git reset/clean/checkout`

L'agent crée **au maximum un checkpoint par répertoire par tour**, donc les sessions longues ne génèrent pas de snapshots excessifs.

9.2 Référence rapide

Commande	Description
<code>/rollback</code>	Lister tous les checkpoints avec statistiques de modifications
<code>/rollback</code>	Restaurer au checkpoint N (annule aussi le dernier tour de chat)
<code>/rollback diff</code>	Prévisualiser le diff entre le checkpoint N et l'état actuel
<code>/rollback</code>	Restaurer un seul fichier depuis le checkpoint N

9.3 Lister les checkpoints

```
/rollback

■ Checkpoints pour /chemin/vers/projet :

1. 4270a8c 2026-03-16 04:36 before patch (1 fichier, +1/-0)
2. eaf4c1f 2026-03-16 04:35 before write_file
3. b3f9d2e 2026-03-16 04:34 before terminal: sed -i s/old/new/ config.py
```

9.4 Prévisualiser avec /rollback diff

```

/rollback diff 1

test.py | 2 +-
1 fichier modifié, 1 insertion(+), 1 suppression(-)

diff --git a/test.py b/test.py
--- a/test.py
+++ b/test.py
@@ -1,1 @@
-print('contenu original')
+print('contenu modifié')

```

9.5 Restaurer avec /rollback

Restaurer vers un checkpoint par son numéro. Hermes effectue les opérations suivantes :

- Vérifie que le commit cible existe dans le dépôt fantôme
- Prend un **snapshot pre-rollback** de l'état actuel (pour pouvoir "annuler l'annulation")
- Restaure les fichiers suivis dans votre répertoire de travail
- Annule le dernier tour de conversation afin que le contexte de l'agent corresponde à l'état du filesystem restauré

```

/rollback 1

■ Restauré au checkpoint 4270a8c5 : before patch
Un snapshot pre-rollback a été sauvegardé automatiquement.
(^_^)b 4 message(s) annulé(s). Supprimé : "Mettre à jour test.py..."
4 message(s) restant(s) dans l'historique.

```

9.6 Restauration d'un seul fichier

```
/rollback 1 src/fichier_casse.py
```

Restaurer uniquement un fichier depuis un checkpoint sans affecter le reste du répertoire.

9.7 Configuration

```

checkpoints:
enabled: true # interrupteur principal (défaut : true)
max_snapshots: 50 # max checkpoints par répertoire

```

9.8 Meilleures pratiques

- Laisser les checkpoints activés — activés par défaut, aucun coût sans modifications de fichiers
- Utiliser /rollback diff avant de restaurer — prévisualiser ce qui va changer pour choisir le bon checkpoint
- Utiliser /rollback plutôt que git reset pour annuler uniquement les modifications pilotées par l'agent
- Combiner avec les git worktrees pour une sécurité maximale — chaque session Hermes dans son propre worktree/branche, avec les checkpoints comme couche supplémentaire

■ Tous les dépôts fantômes résident sous `~/.hermes/checkpoints/`. Chaque sous-répertoire est dérivé du chemin absolu du répertoire de travail. Ces fichiers n'ont pas besoin d'être manipulés manuellement.